NPLify · P0 Technical Baseline

Cutover Approach

New XP Technologies Limited

22 October 2026 · Confidential

NPL runs on spreadsheets today. The plan is to start using NPLify on a staging instance: after every change to a spreadsheet the team makes the same change in the app, for three to four weeks, to see that things go in the right direction; the balances of the opening date are entered as verified opening entries, and the deals open on that date in their current state (Section 4); when everything is right the real projects' data is extracted to production and the production app is used from a quiet day, chosen a couple of days before. This document gives that plan its steps, its checks and its sign-off.

How to read this pack

Four map pages are now part of the pack as documents of their own, and are the place to start: the Top-Down Map (from the business, through the data model and the deal states, to the scenarios and the reading order), the Deal Flow on one page (Quote, Collect, Convert, Settle, with the roles and the money path), the Workflow Tree (8 classes, 27 sub-classes, 92 procedures, each naming who initiates, who approves, the codes it is written under and the records it touches) and the Workflow Tree on one page. Read them in that order, then this document. Every worked transaction (T), acceptance vector (L, B), walkthrough (W, X), flow (F) and procedure (C, A, D) has one identifier, listed in the Reading Companion's identifier register; the Test Strategy & Coverage document says which test proves each procedure.

1 · The approach in one page

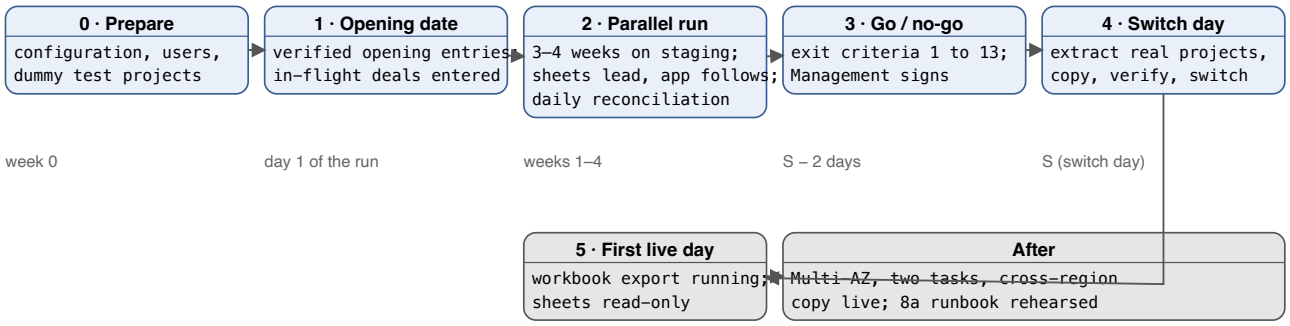
Three things have to be true on the evening of the switch: the configuration in the app describes NPL's business as it is (Configuration Schema), the ledger's balances equal the spreadsheets' balances to the cent, and the team has already run real weeks in the app and trusts it. The plan gets there without touching production until the last day:

1. **Prepare.** Load the configuration (clients, projects, partners, pairs, fee structures, parties, accounts, templates, users and roles) into the staging instance; validate it against the schema rules; fix the spreadsheets' inconsistencies that the validation surfaces before going further. Pick the opening date (Section 4).
2. **Run in parallel on staging.** For three to four weeks the spreadsheets stay the system of record and the app is run beside them: every change made to a sheet is made in the app the same day, by the same person, through the normal screens and approvals. Finance reconciles every evening (Section 5); every difference goes in the discrepancy log and is explained before the next day starts.
3. **Opening entries on the opening date.** On the first day of the parallel run, the opening date chosen by NPL, every balance the sheets show is entered as a verified opening entry: for each project, partner

and currency an opening posting against a dedicated opening-balance account, split by sender within each receiver group, with NPL's pool and anything owed to or by a partner as separate lines; Finance verifies each line against the partner statement or wallet balance of that day, Management approves, and the evidence is attached (Section 4). Deals in flight on the opening date are entered by hand as open deals in their current state. Earlier history stays in the archived sheets, read-only; NPLify's reporting starts on the opening date.

4. **Go / no-go.** The exit criteria of Section 6 are met or the run is extended; Management signs; NPL names the switch day: the day with the least traffic, a couple of days ahead.
5. **The switch day.** Freeze the sheets after the previous day's close; final reconciliation; reset production once, build production from staging by the clean-target extract of Section 7 (the real projects and everything that belongs to them; nothing of the test projects; nothing deleted), verify, point the users at production. The next day is the first live day; the sheets stay read-only for four weeks as the fallback; the staging instance is removed.

Figure 1 — the cutover in six steps, with opening entries in place of the backfill



Changed since: the backfill of earlier deals is gone; the balances of the opening date enter as verified opening entries (Section 4, Ledger Pattern R), so NPL chooses its opening date rather than waiting for a clean break. Scripted test deals live in flagged dummy projects that the extract leaves behind (Section 3). Only steps 3 and 4 touch productio

Figure 1 — The cutover in six steps. Only steps 3 and 4 touch production; everything before them is on the staging instance and reversible.

> **What the parallel run proves, and what it does not.** It proves that the configuration prices and routes NPL's real deals the way NPL does, that the postings of each day reproduce the sheets' balances, and that the team can run its day in the app. It does not prove the ledger's arithmetic — that is the Calculation Specification's acceptance vectors and the Ledger Posting Design's worked transactions, which the build already passes — and it does not test every pattern of the Scenario Catalogue: the weeks will bring whichever deals they bring. Patterns that do not occur in the run are exercised once each on staging as scripted test deals before the go/no-go (Section 6).

Scripted test deals never touch a real project. Every real project has a dummy twin project on staging, set up from the same archetype with the same configuration and flagged as a test project (PROJECT.is_test); the scripted deals that exercise patterns the weeks did not bring are entered there. Nothing is deleted to keep them out: staging's ledger is append-only like production's, and the test projects' postings stay in it. Production is instead built by a clean-target extract (Section 7): starting from the real projects (PROJECT.is_test false), it copies every row that belongs to one of them, following the foreign keys outward, and reads nothing that belongs to a test project or a test user. Staging is kept read-only afterwards as the record of the run, test rows included; the switch-day checklist confirms that no row in production links to a test project and that the real projects' balances equal staging's.

2 · Starting point: the spreadsheets

NPL's workbooks are the source of every figure the cutover loads, and the yardstick of every check. Two kinds of content are taken from them:

In the sheets	Becomes in NPLify	Notes
Master data	CLIENT, PROJECT, PARTNER (with cutoffs and holidays), CURRENCY_PAIR and rate chains, FEE_STRUCTURE and FEE_TIER per partner × pair, FEE_OVERRIDE, SENDER with allow-lists, RECEIVER_GROUP, RECEIVER, RECEIVING_ENTITY and accounts, PARTNER_ENTITY and endpoints, DESTINATION_APPROVAL, MARKUP_SHARE and PARTNER_REBATE rules, MESSAGE_TEMPLATE, SYSTEM_SETTING, USER / ROLE / ROLE_RIGHT	The Project Context and Payment methods tabs, the fee table and NPL's answers of 8 October are the sources; the Configuration Schema's rules are the validation.
Deals	Open on the opening date → entered through the normal screens as open deals in their current state (collected but not converted, converted but not settled, instructed but not confirmed), with the sheet's deal reference and evidence. From the opening date on → entered day by day during the parallel run. Closed before the opening date → not entered; their effect is in the opening entries and the sheets remain their record.	The sheet's deal reference becomes the deal's client reference, so a row in the sheet and a deal in the app are the same thing by name.

The validation will surface things the sheets never had to state: a receiver with no approved destination at a partner, a sender with no allow-list, a fee structure whose parts do not add up, a partner with a bank cutoff but no time zone, a pair with no rate source. Each is resolved with NPL before the run — the run must start from a configuration that passes.

3 · The staging instance and the rules of the parallel run

Staging is a second copy of the production stack in NPL's AWS account (Architecture & Stack Note, Environments): the same image, its own database and bucket, its own sign-in. It exists for the weeks of the run and is removed after go-live. Four rules make the run meaningful:

1. **The sheets lead, the app follows — the same day.** Whoever changes a sheet makes the same change in the app before the end of their shift, as the person they are (Operations enters, Finance approves, Management escalates), through the normal screens. No bulk entry, no "Finance types everything": the point is that the roles and approvals are exercised.
2. **Real values, real timestamps.** The partner's rate as quoted, the market rate at the time (the app captures its own; the sheet's rate is a check), the real cutoff, the real evidence attached. Where the app's clock would have expired a quote the sheet kept alive, the discrepancy log says so: that is a finding about policy, not an error.
3. **Outside services point at copies.** Rates, hash checks and the authenticator app work as in production. The client journal writes to a *copy* of each client's sheet, never to the client's real sheet,

until go-live. No message rendered on staging is sent to a sender or receiver; quote packages are rendered and filed only.

4. **Finance reconciles every evening** with the checks of Section 5, before the next day's first deal. A difference is either fixed in the app (an entry missed or mis-entered), fixed in the sheet (the sheet was wrong — these are findings NPL wants), or explained as a known difference of method (rounding at the pair's unit, a fee part the sheet computes differently) and recorded. A difference that cannot be explained stops the run until it is.

Two things the staging weeks are not. They are not training on test data — that happens on the local environment before the run starts, with the Ledger Posting Design's worked transactions as the exercises. And they are not optional for Finance: the opening entries' sign-off and the daily reconciliation are Finance's work, and the go/no-go is signed on Finance's log.

4 · Opening entries

NPL's decision of 20 October: the balances the sheets show on the opening date are entered as verified opening entries, not reconstructed by re-entering months of deals. The ledger stays append-only and every later balance still derives from postings; the opening entries are postings like any other, with evidence attached, and the history before the opening date stays in the archived sheets. NPL can therefore choose its opening date rather than wait for a clean break; the effort depends on the number of senders with a balance on that day, which Finance has from the sheets, not on the number of historical transactions.

Step	Who	What
1. Choose the opening date	NPL (Management)	The first day of the parallel run. The sheets are reconciled as of the close of the previous day; that reconciled state is what the opening entries reproduce.
2. List every open balance	Finance	Per project, partner and currency: client money collected and not yet converted; converted and waiting to be paid out, split by sender within each receiver group; sender credits; shortfalls owed to receivers; NPL's pool at the partner; rebates and markup shares accrued and not yet settled; loss recoveries agreed and not received; open invoices and their discharged parts (NPL-GR); the month-to-date tier volume of each tiered project.
3. Verify each line	Finance	Against the partner's statement or the wallet's on-chain balance for the opening date, and against the sheet. The statement or screenshot is the evidence. The partner's figure is the control total and the sheets' allocation is the detail. Where the two differ, the whole of the partner's figure is still opened: the part the sheets allocate as the client lines, the difference as a suspense line of the same partner and currency (owner not yet known, so it stays on the client side and never enters NPL's pool), listed on the discrepancy log with Finance as its owner. Nothing is left out because it cannot be allocated.
3a. Resolve what is in suspense	Finance proposes, Management decides	Each suspense line is resolved as one of three things: client money of a named sender and receiver group (moved from suspense to that promise by an opening posting), NPL's own money (moved to the pool, with the evidence that it is NPL's), or a partner error (a query is raised and the line stays until the partner's figure is corrected). The decision is Management's, recorded on the line with its evidence. A suspense line still open at the go/no-go and above the materiality figure Management sets before the opening date (written at the head of the discrepancy log) is a no-go on its own, whatever the rest of the opening journal shows (Section 6, criterion 6). Below that figure Management may accept the line into production with the reason recorded; it then stays on the evening reconciliation until it is cleared.
4. Approve	Management	The whole set per project is one approval (approver not initiator); each line is posted with source Opening, the evidence reference and the sheet row it reproduces.

Step	Who	What
5. Post	Finance (the engine posts)	Ledger Pattern R: client balances post between the client's jar at the partner and the project's promise, so they need no opening account; NPL's own balances and anything owed to or by a partner or party post against the opening-balance account CO.NONE.<CCY>.OPENING_BALANCE (Ledger Posting Design, Section 3). Tier counters and accruals get one line each. A suspense line posts the money at the partner against a suspense account of the same partner and currency (CL.<partner>.<CCY>.SUSPENSE, an account kind to be added to the ERD's list in its next edition), so the partner's one-balance identity holds from day one: the statement equals the client lines plus suspense plus NPL's pool.
6. Enter the in-flight deals	Operations, Finance	Each deal open on the opening date is entered as an open deal in its current state with its reference and evidence, so its remaining steps run in the app; its already-posted part is in the opening entries, not entered again. The steps a deal had completed by the opening date are recorded on it as completed, with the sheet's date and evidence, and the engine posts nothing for a recorded step: only the next step posts. The table after this one says, state by state, what the opening entry holds, what comes next and what is therefore never posted again.
7. Check	Finance	The checks of Section 5 are run against the sheets as of the opening date and must agree exactly before the first live-mirrored deal; Finance signs the opening entries off.

Manual entry on the real screens is kept for the in-flight deals and for the deals of the parallel run itself, which is where the team exercises the workflows and approvals. The proposal already places approved opening entries at M11.

Per-state opening mapping: no step posts twice

State on the opening date	What the opening entry holds	Entered as	Next event, and what it posts	Never posted again
Collecting (part of the money received)	The parts received, into COLLECTED against the project's promise, per sender	A deal in Collecting; the parts received recorded as verified on their sheet dates; the outstanding parts open	The remaining parts are verified and post as a normal collection; the conversion then runs on the whole	The collection of the parts already received
Collected, not converted	The full amount into COLLECTED against the promise	A deal in Collected, its collection recorded as verified	The conversion, posted as normal: COLLECTED to DUE, the fee part, NPL's margin; it consumes the opening COLLECTED balance	The collection

State on the opening date	What the opening entry holds	Entered as	Next event, and what it posts	Never posted again
Converted, waiting to be paid out	DUE against the promise (T60), split by sender; the fee taken at conversion and NPL's margin are already inside the pool line (T61), not in EARN_GROSS	A deal in Converted, with the rate version and fee of the sheet stamped on it	The settlement: release discharges the promise against DUE as normal; confirmation closes the line	The collection, the conversion, the fee and the margin
Sent, not yet confirmed	Nothing: the release already discharged the promise in the sheet, so the amount is in neither opening balance; where the partner still shows the money, rule 13a of the Ledger Posting Design names the state	A deal in Settling with the line in state sent, released_at and expected_confirmation_by from the sheet	The confirmation, which posts nothing; a return follows the bank-return pattern	The release
Sender credit outstanding	The credit into the sender's CREDIT, with the money at the partner on the other side	No deal; the credit is on the sender	The sender's next quote applies the credit, as the Calculation Specification says	The short payout or void that created the credit
Shortfall owed to a receiver	Into SHORTFALL for the receiver group	No deal; the shortfall is on the group	The make-good on the group's next settlement	The shortfall itself
Client money resting in NPL's own wallet	Into HELD for the project, against the promise	The deal in its custody state, with the reroute that put it there recorded	The onward settlement from the own wallet	The move into custody
Rebate or markup share accrued, not settled	REBATE_RECEIVABLE, SHARE_RECEIVABLE or SHARE_PAYABLE against OPENING_BALANCE; the month-to-date tier volume as a counter line, not a posting	No deal; the counter starts at the opened figure	The partner's payment settles the receivable; the next deal adds to the counter from the opened figure and its accrual covers new volume only	The accrual for volume before the opening date
Loss recovery agreed, not received	LOSS_RECEIVABLE against OPENING_BALANCE	The loss event recorded as agreed, with its evidence	The receipt settles the receivable	The loss event's own posting
Open invoice (NPL-GR)	VENDOR_PAYABLE or RESALE_RECEIVABLE against OPENING_BALANCE, the cost and revenue being pre-NPLify	The invoice with its discharged parts	The payment or receipt discharges it	The invoice's own posting
Unallocated difference (step 3)	DUE against SUSPENSE for the partner and currency	A suspense line on the discrepancy log	Management's decision in step 3a moves it to a promise, to the pool or back to the partner	Nothing: it is cleared, never re-opened

The link between a deal entered in flight and the opening entry that holds its money is a reference on the deal's step records (to be added to the ERD in its next edition). The cutover test of the Test Strategy &

Coverage document (G-OPEN) imports one deal in each of these states, continues each to its close and checks that every balance changes exactly once.

5 · Reconciliation checks

Run by Finance every evening of the parallel run, once the opening entries are posted, and once more before go-live against the frozen sheets. Each check names what the app shows, what the sheet shows and the tolerance; a check outside tolerance is a discrepancy-log entry with an owner and a resolution before the next day.

Check	In the app	In the sheets	Tolerance	Owner
Balance at each partner, per currency	PROJECT_BALANCE per project × partner × currency	The sheet's partner balance for the same project and currency	Exact	Finance
Money collected and not yet converted	Σ CL.<PARTNER>.<CCY>.COLLECTED	Open collections on the sheet	Exact	Finance
Owed to receivers, per group	GROUP_ENTITLEMENT	The sheet's outstanding payouts by receiver	Exact	Finance
Open deals and their states	DEAL_GROUP / DEAL lists by state	The sheet's deal register	Same set of references; states agree	Operations
Today's deals	Every deal group created today	Every row added to the sheet today	One-to-one, same amounts, same partner	Operations
Sender credits and shortfalls	CL.<SENDER>.CREDIT, CL.<RECEIVER>.SHORTFALL	The sheet's notes column	Exact	Finance
NPL's money at partners and in own wallets	CO.<PARTNER>.POOL, CO.OWN.WALLET, CUSTODY	The sheet's pool and wallet lines; the wallet's on-chain balance	Exact; on-chain within pending transfers	Finance
Margin of the day	EARN_GROSS postings of the day, per deal	The sheet's margin column (or the Calculation Specification's vectors where the sheet has none)	Within the pair's rounding	Finance
Rebates and shares accrued	PARTNER_REBATE_ACCRUAL, MARKUP_SHARE_ACCRUAL	The sheet's monthly rebate and share working	Exact at month end	Finance
Invoices (NPL-GR)	INVOICE_BALANCE	The reseller workbook's open invoices and instalments	Exact	Finance
Ledger rules	The ledger's balance rules as database checks (balanced per currency and per owner; every balance a sum over postings)	—	Zero violations	Engineering

Check	In the app	In the sheets	Tolerance	Owner
Visibility	A sample of screens and exports per role	Roles & Visibility Matrix	All pass	Engineering with Management

The discrepancy log is a sheet of its own: date, check, project, app figure, sheet figure, cause (app entry · sheet error · method · policy), resolution, who closed it. It is the evidence the go/no-go is signed on, and it is kept after go-live.

6 · Exit criteria and the go / no-go

1. **Ten consecutive business days** with every check of Section 5 inside tolerance at the evening reconciliation, and no discrepancy open for more than one day during the run.
2. **Every project NPL runs today** has had at least one deal through the app during the run, or a scripted test deal on staging where the weeks brought none (the dormant Evo products need none).
3. **The ten flows of the Scenario Catalogue** have each been exercised at least once on staging, by a real deal or a scripted one: F1 Cash by token (collection or settlement); F2 USDT collection with wallet screening; F3 Receiver wallet screened before settlement; F4 SWIFT collection; F5 SWIFT settlement; F6 Returned transfer; F7 One-time local deposit account; F8 Local payouts for small amounts; F9 Bank cutoff; F10 Internal transfer inside the partner's platform.
4. **Every role has done its own work** in the app: Operations has quoted, verified, instructed and prepared; Finance has approved, confirmed rate versions, reconciled and recorded a recovery or an offset; Management has approved an escalation, a configuration change and a user change.
5. **The visibility checks** of the Roles & Visibility Matrix pass on staging with NPL's real configuration.
6. **The opening entries** are complete and signed off by Finance: every project's balances in the app equal the sheets' as of the opening date, and every in-flight deal is in the app in its current state; no suspense line above the materiality figure is open (Section 4, step 3a): an unresolved ownership difference is a no-go on its own, even when every other check agrees.
7. **A restore and the extract have been rehearsed:** the staging database restored from a snapshot into a scratch instance, and the ledger checks run there; and staging extracted into a scratch instance by the switch-day procedure (Section 7): no row linked to a test project, the real projects' counts and balances equal to staging's, every evidence reference and ledger hash resolving (Test Strategy G-FILTER).
8. **Management signs** the go/no-go, on Finance's discrepancy log and the engineering checklist, and names the switch day. A no-go extends the run by whole weeks; nothing is rushed into the switch.
9. **Security and access:** an Operations user tries to see NPL's margin, a pool or a ledger balance, and to approve a settlement, through the screens, the API, an export, a notification and the client journal — expected: denied everywhere; nothing changes; no hidden field reaches the browser. Runs it: Engineering. Signs off: Management.
10. **Concurrent actions:** two Operations users edit the same deal; two Finance users approve the same settlement at once; a job is started twice — expected: exactly one succeeds; no duplicate posting; the second is refused with the current state (Lifecycle & Approvals 6a). Runs it: Engineering. Signs off: Engineering lead.
11. **Application outage mid-transaction:** the application is stopped while a collection is being recorded and while a conversion is being instructed — expected: on restart nothing committed is lost, nothing half-done is posted, the deal shows its true state (Ledger Pattern P). Runs it: Engineering. Signs off: Operations.

12. **Duplicate submission:** the same settlement instruction is sent twice because the first response was slow; the same collection is recorded twice — expected: the second is refused by its idempotency key; one posting. Runs it: Engineering. Signs off: Finance.
13. **Restore and recover:** the database is restored to a point in time and the deals processed after that point are re-entered and reconciled under the recovery runbook (8a) — expected: no missing and no duplicate movement; the identities of the Ledger Posting Design hold; technical restore and resumption are two recorded sign-offs. Runs it: Engineering with Finance. Signs off: Management.

A basic vulnerability assessment of the staging environment is run before go-live, separately from UAT; the penetration test remains the optional add-on of the proposal, for NPL to decide (Architecture note, security operations).

7 · The switch day

The switch day is the day with the least traffic, chosen by NPL a couple of days before once the go/no-go is signed. Nothing below depends on which day of the week it is.

When	Who	What
The day before, close	Operations, Finance	Last entries mirrored; the sheets are frozen (read-only copy made); the app's day is closed with the evening reconciliation.
Switch day, morning	Finance	Final reconciliation of every check against the frozen sheets; the discrepancy log closed.
Switch day, midday	Engineering	Production reset once (the single reset of NPL's architecture page): database dropped and recreated empty, bucket emptied of test runs. Staging put in read-only mode. The clean-target extract run (next row); migrations confirmed at the same schema version; the ledger checks run on production; row counts and balance totals compared with staging — identical, or the extract is repeated.
Switch day, afternoon	Finance, Management	Sign in to production: one screen per role compared with staging; balances per partner, entitlement per group, custody and the exception list match the morning's reconciliation. Client journal switched from the copies to the clients' real sheets. Management records the go-live in the audit log.
Next day	Everyone	First live day in production. The sheets are not written to; whoever reaches for one records the item in the app instead. Finance reconciles that evening against the frozen sheets plus the day's deals, as during the run.
After	Engineering	Staging removed after two clean weeks in production (its last snapshot kept 30 days). The frozen sheets stay read-only for four weeks as the fallback, then are archived.

When	Who	What
Switch day, midday, the extract	Engineering	The extract, in one transaction into the empty production database: (1) the set of real projects, PROJECT.is_test false; (2) every row that belongs to one of them, following the foreign keys outward from PROJECT (client, configuration versions, parties, deals with their legs and parts, quotes, approvals, settlement lines, reroutes, events, flags, locks, opening entries), and the rows shared by every project (partners, rates, templates, settings, the real users and roles); (3) every LEDGER_TRANSACTION with its lines whose deal, opening entry or subject is in that set, copied whole with its id, order and hash, so the chain of the real projects is unchanged; (4) the evidence objects the extracted rows reference, copied by key from staging's bucket, and the audit trail of the extracted rows, copied whole. A row of a test project is never read, and nothing is deleted anywhere: staging keeps its ledger and goes read-only. Recorded: the counts per table extracted and left behind, and the total of the test projects' balances left behind.
Switch day, midday, after the extract	Engineering, Finance	Checklist lines added: no row in any table links to a test project, checked through the foreign keys and not only on PROJECT and DEAL; the count per table equals staging's count of real-project rows; every balance of the real projects equals staging's, and Finance reconciles them against the frozen sheets and the partner figures independently of the extract; every evidence reference and every ledger hash resolves; the opening entries are present with their evidence; the migration numbers and ERD edition match the release note.
Switch day	Engineering	Production runs on the Multi-AZ database and two application tasks in two zones (Architecture note, Sections 6 and 7); the daily cross-region backup copy is confirmed; the fallback workbook export is running (Section 8a).

7a · Handover list

What NPL holds, and what New XP hands over, so that the platform is NPL's to run and to have maintained beyond one developer. The proposal already gives NPL the intellectual property and repository and database access during the build; this list makes the handover a checklist, completed in Phase 3 (M11) and confirmed at go-live.

Item	Where it lives	Confirmed by
Source repository and the CI pipeline	In NPL's own GitHub (or equivalent) organisation from the start of the build; New XP engineers work in it with time-bound access	NPL Management
Infrastructure as code	The AWS setup (tasks, database, bucket, secrets, alarms, backups, cross-region copy) as code in the same repository, so the environment can be rebuilt from it	Engineering
Deployment runbook	How a release is built, tested, applied (migrations, Section 7 of this document) and rolled back	Engineering, Operations
Restore runbook	The recovery runbook of 8a, with the yearly rehearsal date	Finance, Management
Access register	Every privileged holder and why (Architecture note 6a); NPL holds the account root and owner roles	NPL Management
On-call contacts	The on-call engineer and a second named New XP engineer on the support bundle, with the escalation path of Architecture note 6a	NPL Management

Item	Where it lives	Confirmed by
Operating documents	This pack at its go-live edition, the Operating Procedures and the Test Strategy & Coverage document, kept in the repository with the code	New XP
Support bundle	Written approval of the support bundle (Schedule D) before go-live, naming the two engineers	NPL Management

8 · Rollback

Until the switch day, rollback is nothing: staging is discarded or the run extended. On the switch day, if the extract or the verification fails, production is left empty, staging goes back to read-write and the run continues; a new switch day is chosen. After the first live day, if the app cannot be used for a day, the team works from the frozen sheets plus the app's entries so far (the Architecture note's fallback) and enters the day into the app afterwards with the real timestamps in the evidence; the ledger is never edited to catch up, it is posted. A restore from backup is the last resort and a Management decision: the ledger is then the ledger as of the restore minute, and the gap is re-entered from the sheets.

8a · Recovery runbook, and the recovery objectives

NPL's decisions of 20 October: recovery inside one hour, not four; and three additions that are part of the design, not options. A replica and a workbook reduce how often the runbook is needed; they do not replace it, because re-entry and reconciliation still happen after any fallback.

Objective	Target	Who
Recovery time (RTO)	The application is usable again within one hour of an outage in operating hours	Engineering restores
Recovery point (RPO)	No more than 15 minutes of committed data lost; near zero for a hardware failure, since the standby database takes over without a restore; the evidence bucket is inside the recovery point (S3 versioning)	Engineering
Financial integrity	100 % of actual money movements accounted for after any recovery, before unrestricted processing resumes	Finance validates, Management authorises

Design element	What it is	When
Multi-AZ database	A hot standby of the database in a second availability zone with automatic failover; written into Section 6 of the Architecture note with its cost line	Production deployment (M11)
Two application tasks	The single application task becomes two, in two zones, so one zone's failure does not stop the application (Architecture note, Section 7)	M11
Cross-region backup copy	A daily copy of the database backup to a second AWS region, covering a whole-region outage	M11
Fallback workbook	An auto-generated export, refreshed every 15 minutes to a Google Sheet, of open deals and their states, partner balances, the day's confirmed rates, cutoffs and pending approvals, so that the spreadsheet fallback starts from a live snapshot rather than a blank template. NPL restricts access to the Sheet to the people who need it, which handles visibility.	Workbook export in Phase 2 (M7)

The runbook

Step	Who	What
1. Declare	Finance, Engineering	The outage is declared with its time; Operations works from the fallback workbook (Operating Procedures A.12), recording every instruction given to a partner with the deal reference and the time.
2. Restore or fail over	Engineering	A zone failure fails over to the standby automatically. Otherwise Engineering restores to the latest point in time; a restore of production is a Management decision recorded in the audit log. Technical restore is sign-off one.
3. Find the recovery point	Engineering, Finance	The last committed transaction in the restored database is identified and compared with the workbook and the partners' records.
4. Re-enter	Operations, Finance	Everything after the recovery point is re-entered from the workbook and the partner confirmations, in time order, with the real times in the evidence. Re-entry is keyed on the deal reference, so a deal already in the ledger is refused; partner references are stored on every instruction, so an instruction already executed is matched, not repeated. The ledger is never edited to catch up: it is posted.
5. Reconcile	Finance	Every partner's statement or balance is reconciled against the ledger (Section 5 checks; the reconciliation record) before the first new deal.
6. Resume	Management	Resumption of processing is sign-off two, recorded separately from the technical restore, once step 5 agrees and every instruction given during the outage is accounted for.

Rehearsed once in full on staging before go-live (exit criterion 13), and once a year after it.

9 · Roles during the cutover

Role	During the run	On the switch day
Operations	Mirrors every operational change the same day; enters the in-flight deals of the opening date as open deals; reports what the app would not let them do (a finding either way).	Nothing to do; first live entries the next day.
Finance	Approves as Finance in the app, for the day's deals and the in-flight deals; verifies the opening entries and reconciles every evening; owns the discrepancy log; signs the opening entries off.	The final reconciliation; the afternoon sign-in check.
Management	Approves escalations and configuration in the app as Management; reads the log weekly.	Signs the go/no-go and names the day; records the go-live.
New XP engineering	Runs staging; loads and validates the configuration; runs the ledger checks nightly; fixes what the run finds, as releases to staging only.	The reset, the extract, the verification, the switch; on call the next day.

10 · Assumptions and decisions for NPL

1. **Three to four weeks** is the planned length of the run; the exit criteria, not the calendar, end it.
2. **The opening date** is the first day of the parallel run, chosen by NPL; the sheets are reconciled as of the previous close and every open balance on that date is entered as a verified opening entry (Section 4). There is no backfill start date.
3. **Opening entries are posted** from a Finance-verified, Management-approved list, with evidence; in-flight deals and the deals of the run are entered through the screens. No historical deals are re-entered, so the volume of history does not affect the run.
4. **The one reset of production** is the one on NPL's architecture page; the extract from staging means production starts from the reconciled run's real data, then live data after; staging stays as the read-only record of the run, test projects included.
5. **Client journals** write to copies during the run and to the real sheets from the switch day.
6. **The switch day** is NPL's choice, a couple of days ahead, on the least traffic; the runbook of Section 7 does not depend on the weekday.
7. **Staging cost**: a second small service and database for the weeks of the run, about the running cost again (Architecture & Stack Note).